
Git Interview Questions

50 essential questions with concise answers, from basics to slightly advanced

Section	Topic	Questions
A	Git Basics	Q1 to Q15
B	Branching and Merging	Q16 to Q28
C	Remote Repositories and Collaboration	Q29 to Q38
D	Undoing Changes and History	Q39 to Q46
E	Slightly Advanced	Q47 to Q50

Section A: Git Basics

Q1. What is Git?

Ans: Git is a free, open-source distributed version control system that tracks changes in files and lets many people work on the same project together.

Q2. What is a version control system (VCS)?

Ans: A VCS records changes to files over time so you can recall earlier versions, compare changes, and see who changed what and when.

Q3. What is the difference between Git and GitHub?

Ans: Git is the version control tool that runs on your computer. GitHub is an online hosting service for Git repositories that adds collaboration features like pull requests and issues.

Q4. What does 'distributed' mean in Git?

Ans: Every developer has a full copy of the repository, including its complete history, so work (commits, logs, diffs) can be done offline.

Q5. What is a repository?

Ans: A repository (repo) is a project folder tracked by Git. It contains your files plus a hidden `.git` directory that stores the full history.

Q6. How do you create a new Git repository?

Ans: Run `git init` inside a folder. This creates the `.git` directory and starts tracking.

Q7. How do you copy an existing remote repository?

Ans: Use `git clone <url>`. It downloads the whole repository with its history and sets up `origin` as the remote.

Q8. What are the three main areas in Git?

Ans: The working directory (where you edit files), the staging area or index (changes prepared for the next commit), and the repository (committed history).

Q9. What is the staging area?

Ans: An intermediate area where you collect the exact changes you want in your next commit. You add changes to it with `git add`.

Q10. What does `git add` do?

Ans: It moves changes from the working directory into the staging area. `git add .` stages all changes in the current directory.

Q11. What is a commit?

Ans: A commit is a saved snapshot of the staged changes, with a unique ID (SHA hash), an author, a date, and a message.

Q12. How do you make a commit?

Ans: Run `git commit -m "your message"`. Use `git commit -am "msg"` to stage and commit already-tracked files in one step.

Q13. What does `git status` show?

Ans: It shows the current branch, which files are staged, which are modified but not staged, and which are untracked.

Q14. How do you see the commit history?

Ans: Use `git log`. For a compact view use `git log --oneline`, and add `--graph --all` to see branches.

Q15. What is the `.gitignore` file?

Ans: A file listing patterns of files and folders Git should not track, such as `node_modules/`, `*.log`, or `.env`.

Section B: Branching and Merging

Q16. What is a branch in Git?

Ans: A branch is a lightweight, movable pointer to a commit. It lets you work on features or fixes separately from the main line of code.

Q17. How do you create and switch to a new branch?

Ans: Use `git checkout -b feature` or the newer `git switch -c feature`.

Q18. How do you list all branches?

Ans: `git branch` lists local branches, `git branch -r` lists remote branches, and `git branch -a` lists both.

Q19. How do you delete a branch?

Ans: Use `git branch -d name` for a merged branch, or `git branch -D name` to force delete. Delete a remote branch with `git push origin --delete name`.

Q20. What is HEAD?

Ans: HEAD is a pointer to your current position, usually the latest commit of the branch you have checked out.

Q21. What is a detached HEAD?

Ans: It happens when HEAD points directly to a commit instead of a branch (for example after `git checkout <commit-id>`). New commits here can be lost unless you create a branch.

Q22. How do you merge a branch?

Ans: Switch to the target branch, then run `git merge feature`. This brings the feature branch's changes into the current branch.

Q23. What is a fast-forward merge?

Ans: When the target branch has no new commits since the feature branch was created, Git simply moves the pointer forward. No merge commit is created.

Q24. What is a three-way merge?

Ans: When both branches have new commits, Git combines them using the common ancestor and the two branch tips, and creates a new merge commit.

Q25. What is a merge conflict?

Ans: It occurs when the same lines of a file were changed differently in both branches, so Git cannot decide automatically which version to keep.

Q26. How do you resolve a merge conflict?

Ans: Open the conflicted files, edit the sections between the `<<<<<<`, `=====` and `>>>>>>` markers, then `git add` the files and `git commit`.

Q27. What is `git rebase`?

Ans: Rebase moves or replays your branch's commits on top of another branch, producing a straight, linear history.

Q28. What is the difference between merge and rebase?

Ans: Merge keeps the full history and adds a merge commit. Rebase rewrites history to be linear. Avoid rebasing commits already pushed to a shared branch.

Section C: Remote Repositories and Collaboration

Q29. What is a remote in Git?

Ans: A remote is a named link to a repository hosted elsewhere, such as on GitHub. View remotes with `git remote -v`.

Q30. What is `origin`?

Ans: `origin` is the default name Git gives to the remote repository you cloned from.

Q31. How do you add a remote?

Ans: Use `git remote add origin <url>`.

Q32. What does `git push` do?

Ans: It uploads your local commits to a remote branch, e.g. `git push origin main`. Use `git push -u origin main` the first time to set upstream tracking.

Q33. What is the difference between `git fetch` and `git pull`?

Ans: `git fetch` downloads new data from the remote but does not change your files. `git pull` does a fetch followed by a merge (or rebase) into your current branch.

Q34. What is a pull request (PR)?

Ans: A request on platforms like GitHub asking maintainers to review and merge changes from one branch into another. GitLab calls it a merge request.

Q35. What is a fork?

Ans: A fork is your own server-side copy of someone else's repository, used to make changes and propose them back via a pull request.

Q36. What is an upstream branch?

Ans: The remote branch that a local branch tracks, so `git push` and `git pull` know where to go without extra arguments.

Q37. Why does `git push` get rejected sometimes?

Ans: Usually because the remote has commits you do not have. Run `git pull` (or fetch and rebase) first, resolve any conflicts, and push again.

Q38. What is a tag in Git?

Ans: A tag marks a specific commit, usually a release like `v1.0`. Create one with `git tag v1.0` and push tags with `git push --tags`.

Section D: Undoing Changes and History

Q39. How do you discard changes in a file you have not staged?

Ans: Use `git restore file.txt` (or the older `git checkout -- file.txt`).

Q40. How do you unstage a file?

Ans: Use `git restore --staged file.txt` (or `git reset HEAD file.txt`).

Q41. How do you change the last commit message?

Ans: Use `git commit --amend -m "new message"`. Only do this if the commit has not been pushed yet.

Q42. What is `git reset`? Explain its modes.

Ans: It moves the branch pointer to an earlier commit. `--soft` keeps changes staged, `--mixed` (default) keeps changes unstaged, and `--hard` deletes the changes completely.

Q43. What is `git revert`?

Ans: It creates a new commit that undoes the changes of an earlier commit, without rewriting history. It is safe to use on shared branches.

Q44. What is the difference between `git reset` and `git revert`?

Ans: Reset rewrites history by moving the branch pointer back. Revert keeps history and adds an undo commit. Use revert for commits already pushed.

Q45. What is `git stash`?

Ans: It temporarily saves uncommitted changes and cleans your working directory. Bring them back with `git stash pop`, and view saved stashes with `git stash list`.

Q46. How do you see what changed in files?

Ans: `git diff` shows unstaged changes, `git diff --staged` shows staged changes, and `git show <commit>` shows the changes in one commit.

Section E: Slightly Advanced

Q47. What is `git cherry-pick`?

Ans: It applies the changes from one specific commit onto the current branch: `git cherry-pick <commit-id>`.

Q48. What is `git reflog` and when is it useful?

Ans: Reflog records every move of HEAD, including resets and deleted branches. It helps you find and recover 'lost' commits.

Q49. What is `git blame`?

Ans: It shows who last changed each line of a file and in which commit, which helps when tracing the origin of a bug.

Q50. What is a good commit message?

Ans: A short summary line (about 50 characters) in the imperative mood, e.g. "Add login validation", optionally followed by a blank line and a detailed explanation.